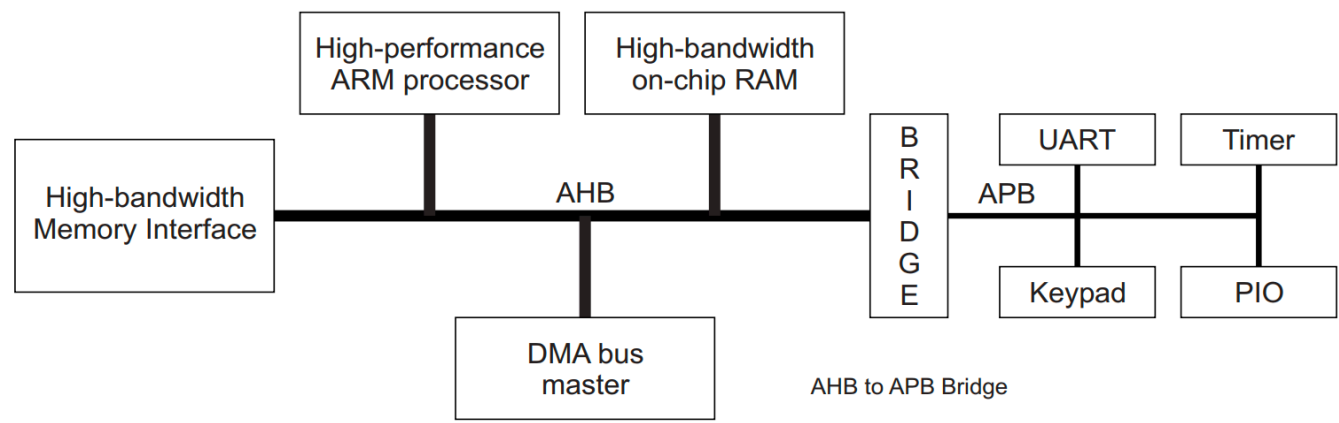


Lab3: 用SystemC建模AMBA AHB总线

DDL: 2025.12.16 23:59:59 (3 weeks)

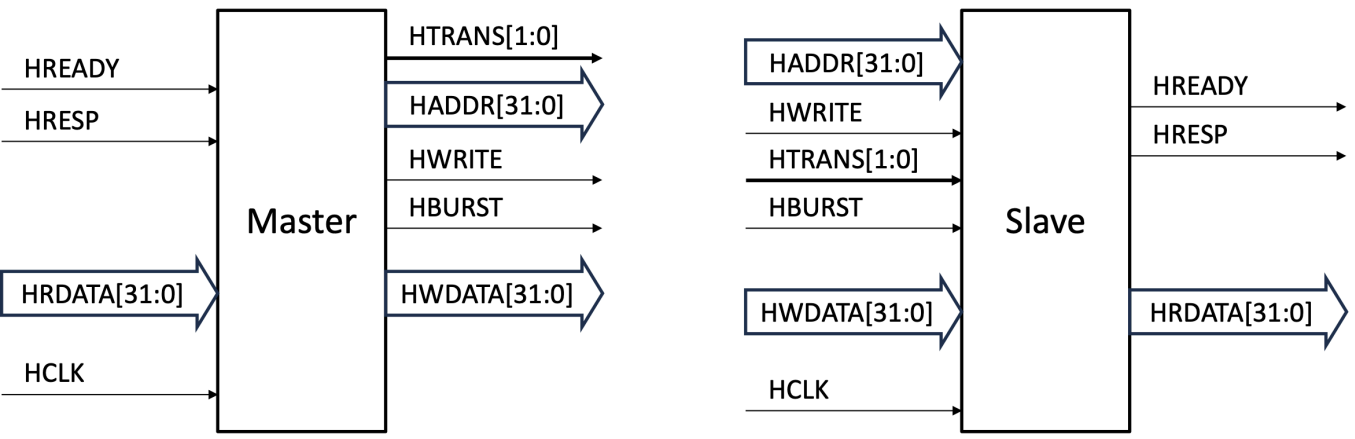
AMBA(Advanced Microcontroller Bus Architecture)定义了一系列片上数据传输的标准用于设计高性能的微处理器，其中AHB(Advanced High-performance Bus)是一种最常用的协议。实际使用到AHB总线的片上系统如下图所示，有CPU、RAM、DMA等单元通过总线互联，还可能通过Bridge和APB总线连接外部的输入输出设备。发出数据读写请求的单元称为master，例如CPU、DMA等，处理数据读写请求的单元称为slave，例如RAM等。



在本次作业中，我们采用SystemC对AMBA AHB总线进行建模。为了方便实现，我们假设系统中只有一个master和一个slave通过总线连接，总线只是简单的将master和slave的信号一一互联。为了简便，slave访问总线的逻辑已经由助教实现，你需要根据下面描述的AHB总线协议，针对不同类型的读写请求实现master访问总线的逻辑。

功能描述

本次作业分为从易到难的三个步骤：（1）实现基本的总线传输操作；（2）实现多个读写请求之间的流水线；（3）实现Burst操作。最终master和slave需要实现的所有输入输出接口如下图所示。



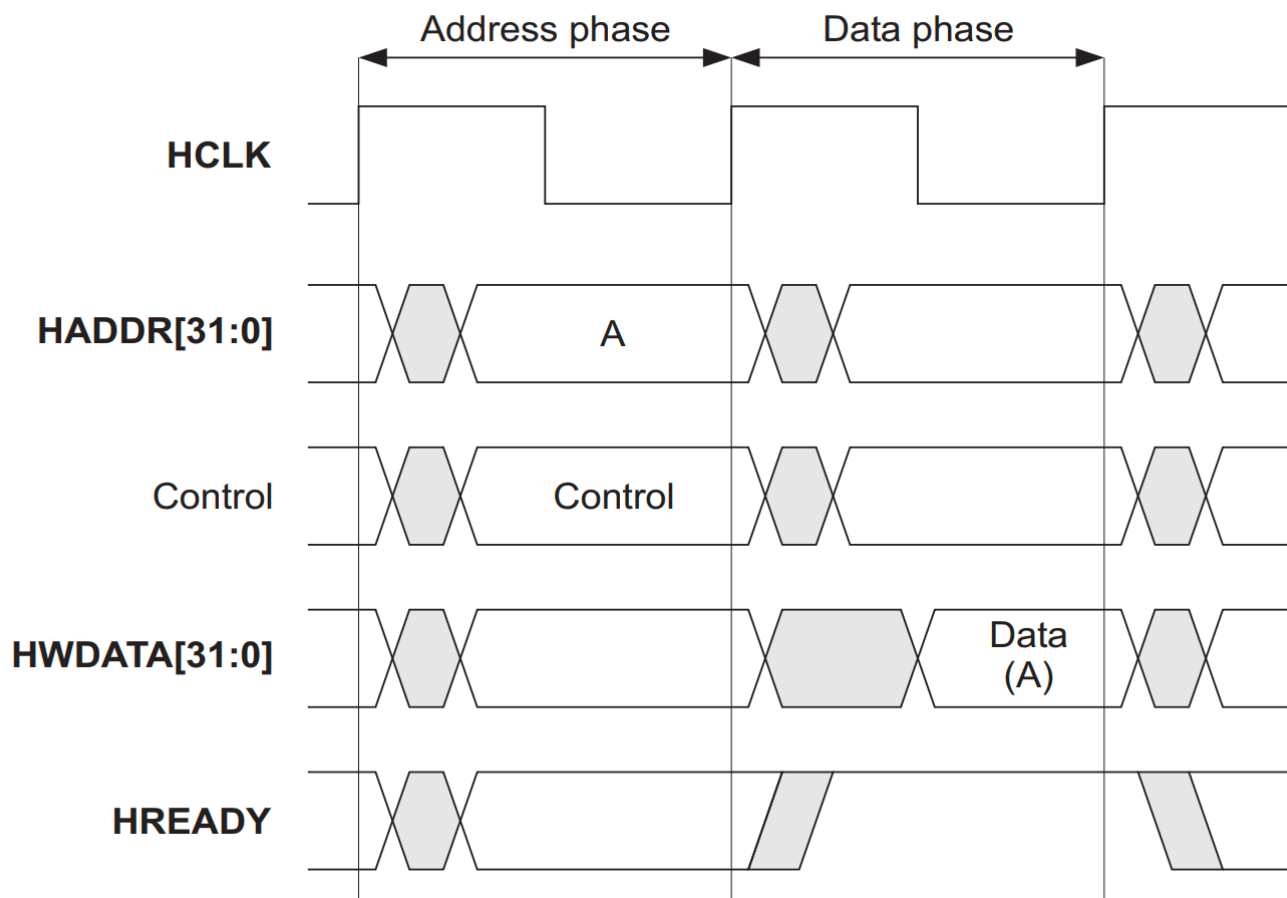
Step 1: 基本的总线传输操作

基本的总线传输操作由两个阶段组成：Address phase和Data phase。在Address phase中，Master发出请求的地址（HADDR）以及相关的控制信号（HTRANS, HWRITE等）。在Data phase中，对于写操作，Master将写的数据发送到HWDATA，对于读操作，Slave将读的数据发送到HRDATA。Address phase一定只有一个周期。Data phase可能有一个或多个周期：如果第一个周期Slave设置的HREADY就是HIGH，那么Data phase将在一个周期中完成；如果第一个周期Slave设置的HREADY是LOW，那么Data phase会直到HREADY为HIGH的周期才完成，对于写操作，Master将HWDATA一直保持为写的数据，直到HREADY为HIGH，对于读操作，只有HREADY为HIGH的周期HRDATA的内容才是有效的。

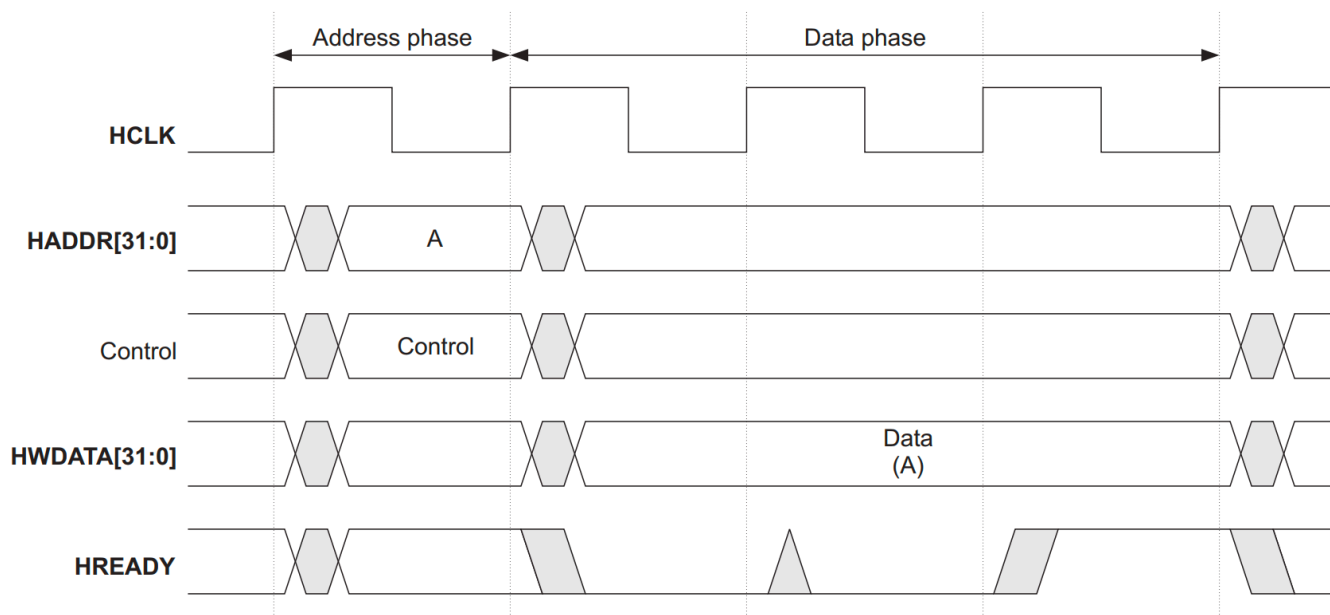
需要用到以下的信号：

信号名称	解释
HCLK	时钟。
HADDR[31:0]	请求的地址。
HWDATA[31:0]	写的数据，固定每次传输4字节。
HRDATA[31:0]	读的数据，固定每次传输4字节。
HREADY	Slave当前周期是否可以读写数据，如果为LOW，那么当前Data phase需要等待直到该信号为HIGH的周期才结束。
HTRANS[1:0]	包含IDLE, BUSY, NONSEQ和SEQ四种状态。IDLE状态表示Master这个周期没有读写请求，NONSEQ状态表示Master在这个周期开启一个新的传输请求（Address phase）。在Data phase中不需要关心这个信号的值。这里暂时不需要关心BUSY和SEQ状态，因为它在Burst操作中才会用到。
HWRITE	为HIGH时表示当前操作为写，为LOW时表示当前操作为读。
HRESP	包含OKAY和ERROR两种状态。ERROR状态是当请求的地址非法时返回的信号。该信号的值在Data phase完成的那个周期是有效的。

当Data phase为一个周期时，示例波形图如下图所示：



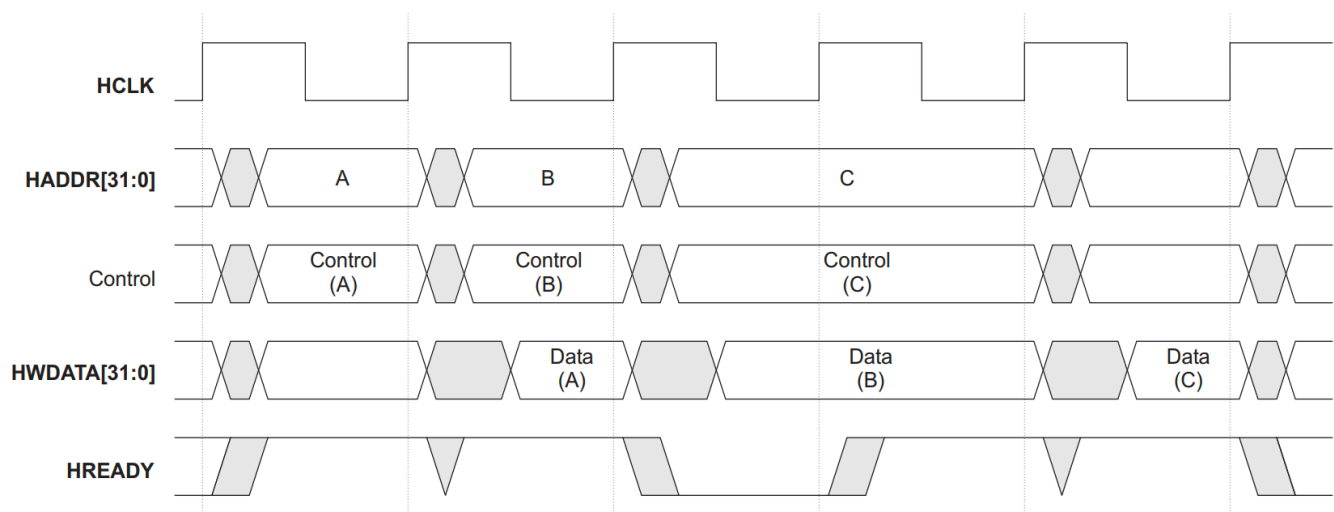
当Data phase为多个周期时，示例波形图如下图所示：



Step 2: 多个读写请求之间的流水线

由于读写请求分为Address phase和Data phase两个阶段，我们可以将多个读写请求进行流水线优化，在第一个请求的Data phase的同时进行第二个请求的Address phase。需要注意的是，如果第一个请求的Data phase由于HREADY为LOW而延迟了若干个周期才完成，那么第二个请求在这若干个周期中也被延迟，HADDR信号和控制信号（HTRANS, HWRITE等）需要一直保持。

一个示例波形图如下图所示：



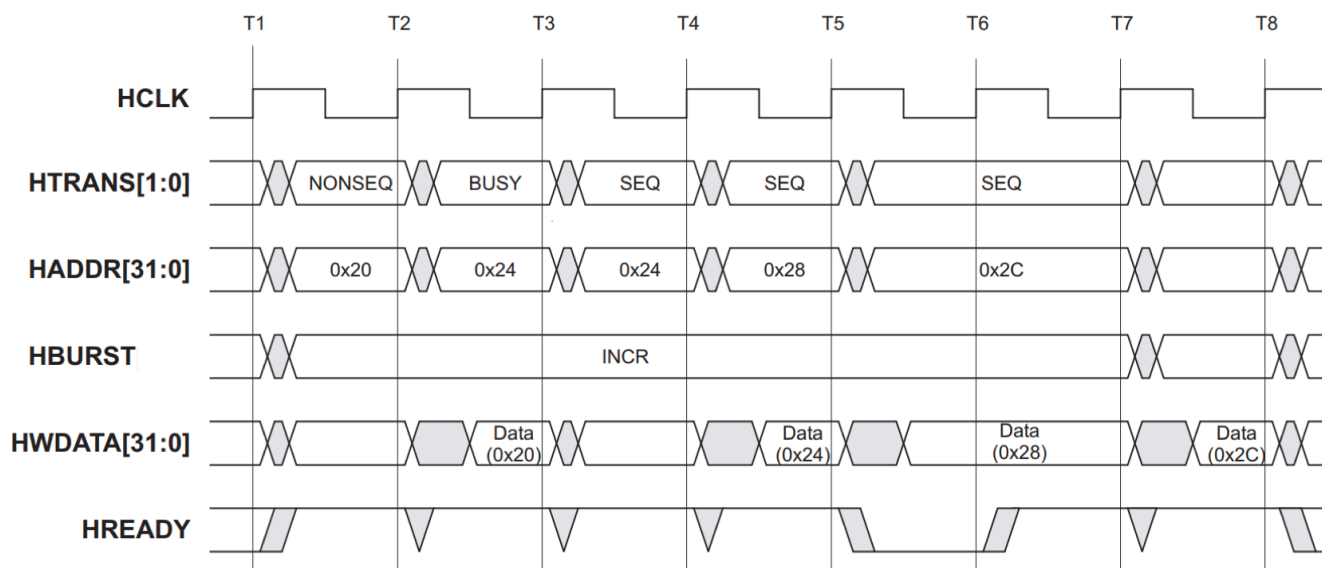
Step 3: Burst操作

Burst操作是指Master连续发送多个对连续地址的读写请求。为简便起见，我们在此处只对其中一种Burst操作进行建模：Master连续发送一定数量的Burst请求，每次固定传输4字节数据。

在这种情况下，需要用到HBURST信号，与单个请求相比含义有变化的信号也在下表中：

信号名称	解释
HADDR[31:0]	请求的地址。在连续的Burst请求中，后一个地址必须等于前一个地址加上4字节。
HTRANS[1:0]	包含IDLE, BUSY, NONSEQ和SEQ四种状态。在连续的Burst请求中，第一个请求的HTRANS为NONSEQ，从第二个请求开始，HTRANS为SEQ。Master可以在连续的Burst请求中间插入BUSY状态，表示在这个周期暂停Burst请求。
HBURST	包含SINGLE和INCR两种状态。SINGLE表示单个请求，INCR表示Burst请求。

一个示例波形图如下图所示。为简便起见，同一个Burst请求内部是如下图所示的流水线形式，但不同Burst请求之间、Burst请求与普通请求之间不需要实现流水线优化。此外，由于一个完整的Burst请求由一次函数调用来实现较为简便，我们假设Burst请求都是连续进行的，中间不暂停，你无需在同一个Burst请求内部考虑插入BUSY状态。



实现方法

我们提供了代码模板，你需要实现的是Master发送读写请求的逻辑：

```
// In master.cpp
void Master::basic_read_request(sc_uint<32> addr, sc_uint<32> &rd_data) {
    // TODO
}
void Master::basic_write_request(sc_uint<32> addr, sc_uint<32> wr_data) {
    // TODO
}
void Master::pipelined_read_request(sc_uint<32> addr, sc_uint<32> &rd_data) {
    // TODO
}
void Master::pipelined_write_request(sc_uint<32> addr, sc_uint<32> wr_data) {
    // TODO
}
void Master::burst_read_request(sc_uint<32> start_addr, sc_uint<32>
burst_len, std::vector<sc_uint<32>> &rd_data) {
    // TODO
}
void Master::burst_write_request(sc_uint<32> start_addr, sc_uint<32>
burst_len, const std::vector<sc_uint<32>> &wr_data) {
    // TODO
}
```

这些函数分别为：

- 基本读写： `basic_read_request` 和 `basic_write_request`

- Pipeline读写: `pipelined_read_request` 和 `pipelined_write_request`
- Burst读写: `burst_read_request` 和 `burst_write_request`

你需要实现这些函数, 根据Master的读写请求, 设置Master的输出接口信号, 并正确处理Master的输入接口信号。

注意:

1. 你可以在Master中添加其他的成员函数和变量辅助实现, 但请不要修改Master中所有输入输出信号和函数的定义, 否则Testbench会报错!!
2. 基本读写与Pipeline读写的区别在于, 基本读写的函数调用需要等到读写完成之后返回, 而Pipeline读写的函数调用会立即返回, 不等待读写完成。在Testbench中会检查两者的实现是否正确。

我们已经实现了Slave处理读写请求的逻辑, 可供参考。Slave读写逻辑如下:

- Slave在处理读和写的时候会有相应的延迟, 具体延迟周期在 `slave.h` 中定义。
- 对于单个读写请求, 延迟周期为HREADY为low的周期数。
- 对于burst读写请求, slave在接收第一个数据时会延迟该周期数, 之后接收数据时无延迟 (保持HREADY为high)。

本次作业的代码模版在服务器 `/root/handout/lab3` 目录下, 请先将代码模版复制到工作目录下。由于助教已经在服务器上配置好了SystemC环境, 你直接执行 `bash run.sh` 即可编译运行。

评分标准

我们使用Testbench随机生成一些读写请求并调用你实现的函数, 检查读写的过程和结果是否正确。

本次作业满分100分。

- 正确实现基本读写操作可得到60分。
- 正确实现基本读写操作和Pipeline读写操作可得到80分。
- 正确实现基本读写操作、Pipeline读写操作和Burst读写操作可得到100分。

目前已公布测试基本读写操作正确性的testbench, 其他testbench会在12.2 (下周二) 公布。

提交方法

最终提交的文件为 `master.h` 和 `master.cpp`。

请把这两个文件放在服务器的 `~/hw/lab3` 目录下, 确保运行 `ls ~/hw/lab3` 时输出如下所示, 不要提交多余的文件。

master.h master.cpp